

Chapter 2 · Lesson 3 — Creating the Data Files

Chapter 2 · Lesson 3 — Creating the Data Files

Goal: fill out `lib/matches.ts`, `lib/teams.ts`, `lib/players.ts`, and `lib/constants.ts`. By the end of this lesson, the data layer is complete and `calculateTeamScores(team)` produces real numbers.

Why this lesson is “boring” and why that’s a feature

You’re about to spend an hour typing match results, team picks, and player metadata. There’s no React in this lesson. No JSX. No clever logic. Just data.

This is **on purpose**. Two reasons:

1. Real apps spend a lot of time on data plumbing.

In an interview, you’ll be asked to build a feature in 45 minutes. **A third of that time is going to be typing fixture data** so you have something to render. Practicing it on a real, larger-than-toy dataset prepares you for that.

2. Typing the data forces you to feel the invariants.

When you write a Team object and have to lay out its 16 R1 picks in the same order as the 16 matches in `ROUND1_MATCHES` — you internalize Lesson 1.2’s invariant in your fingers. That’s worth more than reading about it three times.

If you really want to skip the typing, copy the files from the repo. **But read every section even if you copy.** The commentary explains *why* each piece looks the way it does.

The four files we'll build

```
lib/
├─ matches.ts      ← all 31 matches across 5 rounds
├─ teams.ts        ← all 8 fantasy teams with their picks
├─ players.ts      ← player metadata dictionary
└─ constants.ts    ← shared style helpers and ball colors
```

These are the **inputs** to the scoring function from Chapter 1 Lesson 3. Without them, `calculateTeamScores` has nothing to crunch.

File 1: `lib/matches.ts`

Open `lib/matches.ts` in the repo. The file starts like this:

```
import type { Match } from './types';

export const ROUND1_MATCHES: Match[] = [
  { id: 1, p1: 'Zhao Xintong', p2: 'Liam Highfield', winner: 'Zhao Xintong' },
  { id: 2, p1: 'Judd Trump', p2: 'Gary Wilson', winner: 'Judd Trump' },
  // ... 14 more
];
```

Walk through the structure:

Why one type import at the top

```
import type { Match } from './types';
```

The `import type` keyword is a TypeScript-only thing. It says “I’m only importing this for type-checking; erase this import in the compiled JS.” The runtime never needs `Match` (it’s just a shape, not a value), so erasing it produces smaller bundles.

If you wrote `import { Match }` instead, the compiler would *also* erase it because it’s an interface — but `import type` makes the intent explicit. Senior projects use `import type` consistently.

```
export const ROUND1_MATCHES: Match[]
```

- **export** so other files (`lib/scoring.ts`, components) can import it.
- **const** because we never reassign the array. (We could mutate the array's contents, but TypeScript's `readonly` modifiers can prevent that too — we don't bother for an internal data file.)
- **: Match[]** is the type annotation: an array of `Match` objects. **TypeScript will complain** if you forget a required field on any item, or add a typo, or pass a number where a string is expected. That's the whole point.

The objects themselves

```
{ id: 1, p1: 'Zhao Xintong', p2: 'Liam Highfield', winner: 'Zhao Xintong'
```

Six fields: `id`, `p1`, `p2`, `winner`, `score`, `seed1`. All required for finished R1 matches. Matches the `Match` interface exactly.

The fields are aligned vertically with whitespace for readability — that's a style choice, not a requirement. Prettier will collapse them on save unless your `printWidth` is wide enough; the codebase uses 100 chars and the alignment fits comfortably.

Pending matches look different

By the end of the tournament every match has a winner. But during the tournament, before any QF matches were finished, the `QF_MATCHES` array would have looked like this:

```
{ id: 1, p1: 'Zhao Xintong', p2: 'Mark Allen' }, // no winner, no score yet
```

Two fields instead of five. **Optional fields can simply be left out.** This is the “absent vs zero” distinction from Lesson 1, expressed in data.

In the *current* state of the codebase, every match has been played and every winner is filled in. But you can imagine going back to a pre-Final state where `FINAL_MATCH[0]` was just `{ id: 1, p1: 'Shaun Murphy', p2: 'Wu Yize' }` — no winner, no score. The scoring function would correctly return `null` for everyone's Final pick until the field is filled.

The four other rounds

```
export const ROUND2_MATCHES: Match[] = [ /* 8 entries */ ];
export const QF_MATCHES: Match[]      = [ /* 4 entries */ ];
export const SF_MATCHES: Match[]      = [ /* 2 entries */ ];
export const FINAL_MATCH: Match[]     = [ /* 1 entry  */ ];
```

Same shape, fewer entries. Note `FINAL_MATCH` is still an **array** even though it has just one element. Why?

Because consistency with the other rounds is more valuable than the trivial savings of `export const FINAL_MATCH: Match`. Code that does `.map()` over rounds can treat them all uniformly:

```
[ROUND1_MATCHES, ROUND2_MATCHES, QF_MATCHES, SF_MATCHES, FINAL_MATCH].forEach(
  round.forEach(m => { /* ... */ }));
});
```

If `FINAL_MATCH` were a single `Match` object, the loop above would crash. **Symmetric data shapes simplify the code that reads them.**

How the file ends

The full file is about 90 lines. It exports five constants. The order is **R1 first, Final last** — chronological order from the user's point of view, not bracket-evaluation order. The reader's brain is happier in chronological order.

File 2: `lib/teams.ts`

This is the longest file in the project. Opening it in the repo:

```
import type { Team } from './types';

export const TEAMS: Team[] = [
  {
    name: 'Invincibles',
    color: '#0F5132',
    accent: '#10B981',
    icon: '🏆',
  },
];
```

```

    motto: 'Unstoppable & Unbeatable',
    r1: ['Zhao Xintong', 'Judd Trump', 'John Higgins', 'Mark Williams', /* ... */],
    r2: ['Zhao Xintong', 'Mark Williams', /* ... */],
    qf: ['Zhao Xintong', 'Mark Allen', 'John Higgins', 'Wu Yize'],
    sf: ['Shaun Murphy', 'Wu Yize'],
    final: 'Wu Yize',
  },
  {
    name: 'Uncredibles',
    color: '#DC2626',
    accent: '#F87171',
    // ...
  },
  // ... 6 more teams
];

```

What's in each team

- **Identity:** name, color, accent, icon, motto. The color and accent are the team's primary brand pair, used for gradients in headers and badges.
- **Picks:** r1 (16 strings), r2 (8 strings), qf (4 strings), sf (2 strings), final (1 string).

That's it.

How to type 16 picks without going insane

Realistically you do this with a printout of the bracket on one side and your editor on the other. For each match in `ROUND1_MATCHES`, ask the team owner “*who do you pick?*” and write the name in the corresponding slot.

The order is **the same as** `ROUND1_MATCHES`. So `r1[0]` is the team's pick for `ROUND1_MATCHES[0]` (which is the Zhao vs Highfield match — so the answer is either 'Zhao Xintong' or 'Liam Highfield').

When you're typing the file, having both files open side-by-side in split view is essential. **Visual alignment between two files is the real-world enforcement of the invariant.**

Color choices

Each team has a primary color and an accent. The convention used in this codebase:

- **color** is the darker / saturated shade.
- **accent** is the lighter shade.

They're paired in CSS gradients like:

```
background: `linear-gradient(135deg, ${team.color} 0%, ${team.accent} 100%)`
```

That produces a smooth gradient from dark to light along a 135° diagonal. The visual richness of the team headers comes entirely from this two-color recipe.

When choosing colors:

- Don't use HSL math to pick the accent. Use a tool like Tailwind's color palette (emerald-700 paired with emerald-400, etc.) or coolors.co.
- Avoid pure red/green/blue — they look childish. Stick with curated palettes.
- Make sure the contrast is high enough that white text reads cleanly on top.

Two optional fields you might see

Recall from Lesson 1.2 that Team has two optional helper arrays: `r1Players?` and `r2Players?`. You'll see them on a few teams. They're only used in edge cases for scoring. Most of the eight teams will not have these arrays. Don't worry about them yet; you'll meet them again in Chapter 4 if at all.

File 3: lib/players.ts

```
import type { PlayerInfo } from './types';

export const PLAYER_INFO: Record<string, PlayerInfo> = {
  'Zhao Xintong': { country: '🇨🇳', seed: 1, status: 'Defending Champion', f...
  'Judd Trump': { country: '🇬🇧', seed: 2, status: 'World No. 1',
  'Mark Allen': { country: '🇮🇪', seed: 3, status: 'Top Tier',
  // ... 29 more
};
```

This is the **dictionary** we discussed in Lesson 1.2 — keyed by player name, value is meta-data.

Record<string, PlayerInfo> explained

Record<K, V> is TypeScript shorthand for “an object whose keys are of type K and values are of type V.” Record<string, PlayerInfo> means “any string key is allowed; every value must be a PlayerInfo.”

It’s **identical** to { [key: string]: PlayerInfo }. Both forms work. Record reads cleaner.

Country flag emojis

Yes, you can put flag emojis directly in TypeScript strings. UTF-8 encoding handles them natively. Just paste them in:

```
'🇨🇳' // China
'🚩' // Black flag – hopefully not for England
'🏴󠁧󠁢󠁥󠁮󠁧󠁿' // Actual England flag (regional emoji, made of multiple code points)
```

Note: the codebase uses 🚩 (a generic black flag) for English players because the regional emoji 🏴󠁧󠁢󠁥󠁮󠁧󠁿 doesn’t render reliably across browsers/OSes. **Pick what renders** — don’t pick what’s “technically correct” if it looks broken on most devices.

Seeds: top-16 only

```
'Liam Highfield': { country: '🚩', status: 'Qualifier', flag: 'ENG' }, // no
```

Of the 32 players, the top 16 are seeded (1 through 16). The other 16 are qualifiers and have no seed field. The Player Info interface marks seed?: number as optional precisely for this case.

Why a dictionary, not an array

Already covered in Lesson 1.2. The most common operation is “given a player’s name, get their metadata” — that’s an O(1) hash lookup with a dictionary, or an O(n) .find() with an array. The dictionary wins on both performance and readability.

File 4: lib/constants.ts

This is the smallest file but pulls its weight. Open it:

```
// lib/constants.ts

export const BALL_COLORS = [
  '#FFFFFF', // white
  '#FBBF24', // yellow
  '#16A34A', // green
  '#92400E', // brown
  '#3B82F6', // blue
  '#EC4899', // pink
  '#000000', // black
];

export const th: React.CSSProperties = {
  padding: '14px 16px',
  textAlign: 'left',
  fontSize: 13,
  fontWeight: 800,
  letterSpacing: '1px',
};

export const td: React.CSSProperties = {
  padding: '12px 8px',
  borderBottom: '1px solid #F3F4F6',
  fontSize: 15,
};

export function tabStyle(active: boolean): React.CSSProperties {
  return {
    padding: '12px 22px',
    border: 'none',
    cursor: 'pointer',
    borderRadius: 10,
    fontFamily: 'inherit',
  };
}
```

```

fontWeight: 700,
fontSize: 15,
background: active ? '#0F5132' : '#F3F4F6',
color: active ? '#FBBF24' : '#374151',
transition: 'all 0.2s',
};
}

```

Three exports:

BALL_COLORS — the seven snooker ball colors

```
export const BALL_COLORS = ['#FFFFFF', '#FBBF24', '#16A34A', '#92400E', '#3B5998', '#F8766D', '#4F81BD'];
```

Used in the hero header to render seven decorative dots in the top-right corner. Pure aesthetic. Lives in `constants.ts` because (1) the colors are a reusable palette and (2) extracting them out of the orchestrator keeps that file readable.

th and td — shared table cell styles

```
export const th: React.CSSProperties = { padding: '14px 16px', /* ... */ };
export const td: React.CSSProperties = { padding: '12px 8px', /* ... */ };

```

These are used by the standings table. They're typed as `React.CSSProperties` — the type that React expects for `inline style={...}` props. By centralizing them here, every place that renders a table cell can pull from the same source:

```
import { th, td } from '@/lib/constants';

<th style={th}>Name</th>
<td style={td}>Wu Yize</td>

```

tabStyle — a function that returns a style object

```
export function tabStyle(active: boolean): React.CSSProperties { ... }
```

This is a **style helper** — a function that takes a piece of state and returns a style object.

The argument decides which colors come back. Useful when many places need the same “tab pill” look but with different active states.

This is what experienced React engineers do *before* reaching for a CSS-in-JS library. **A function returning a style object is the simplest possible style abstraction.** It’s also TypeScript-friendly.

Sanity-checking with a quick console smoke test

Once all four files are in place, you can verify your data by adding a temporary `console.log` in `lib/scoring.ts`:

```
import { TEAMS } from './teams';

const test = TEAMS.map(t => ({ name: t.name, scores: calculateTeamScores(t) })
console.log(test.sort((a, b) => b.scores.total - a.scores.total));
```

This won’t run in the browser yet (we don’t have a UI), but if you boot a Node REPL or write a quick `node test.ts` script with `tsx`, you should see the eight teams sorted by total score with sensible numbers — not all zeros, not all NaN, not all undefined.

If the numbers look wrong:

- **All zeros?** Probably the match arrays don’t have winner filled in, or the picks don’t match player names exactly (typo).
- **NaN total?** Probably a `pts` value somewhere is undefined; check `scorePick` is reaching all branches.
- **Score is correct but unsorted?** That’s fine — sorting happens in the orchestrator (Chapter 4).

Remove the `console.log` after you’ve sanity-checked. Production code shouldn’t `console.log`.

What you’ve built so far

Three lessons in. Your `lib/` folder is complete:

```
lib/
├─ types.ts      ← (Chapter 1.2)
├─ scoring.ts    ← (Chapter 1.3)
```

```
|— matches.ts      ← (this lesson)
|— teams.ts        ← (this lesson)
|— players.ts      ← (this lesson)
|— constants.ts    ← (this lesson)
```

That's the **entire data layer** of the app. **Zero React**. Zero JSX. Zero DOM. If you handed this folder to someone using a different UI framework — Vue, Svelte, Solid — they could build the same app on top of it without changing a line.

This is the property you want from your data layer. **Framework-independent business logic**. When the React replacement comes along in 2030, your `lib/` folder migrates unchanged.

Vibe prompt you would have used

"Create `lib/matches.ts`, `lib/teams.ts`, and `lib/players.ts` with TypeScript constants. The data is: [paste the bracket and team picks here]. Each match is { `id`, `p1`, `p2`, `winner?`, `score?`, `seed1?` }. Each team has 5 picks arrays in match-index order: `r1[16]`, `r2[8]`, `qf[4]`, `sf[2]`, `final` (single string). `PLAYER_INFO` is a `Record<string, PlayerInfo>` keyed by player name. Use `import type` for the `Match/Team/PlayerInfo` types from `./types`. Do not generate any UI."

Notice the constraint: "Do not generate any UI." For data-only tasks, you say so explicitly. **Without that constraint, the LLM will helpfully make you a `<MatchTable>` component you didn't ask for.**

CHECK YOURSELF

1. **Why is `import type` used instead of `import` for the type imports?** What's actually different in the compiled output?
2. **`FINAL_MATCH` is an array of one element instead of a single `Match` object. Justify the choice.** Would symmetry alone be enough? Are there other reasons?
3. **Of the eight teams, three have ONE typo in their `r1` picks (e.g. 'Zhao Xintong ' with a trailing space).** Walk through what would happen at scoring time. Would the bug be visible immediately? When and how?
4. **A new player joins the bracket as a 17th seed (replacing a withdrawal).** What's the minimum set of files you have to update? Walk through the change.

5. `PLAYER_INFO` uses `Record<string, PlayerInfo>`. The other type form is `{ [key: string]: PlayerInfo }`. They're equivalent. Pick one and explain to a junior dev why you'd prefer it.

When you've answered these, you've finished Chapter 2. Onward to [Chapter 3 – Your First Component](#), where we **finally** import React.